Memory Management 1

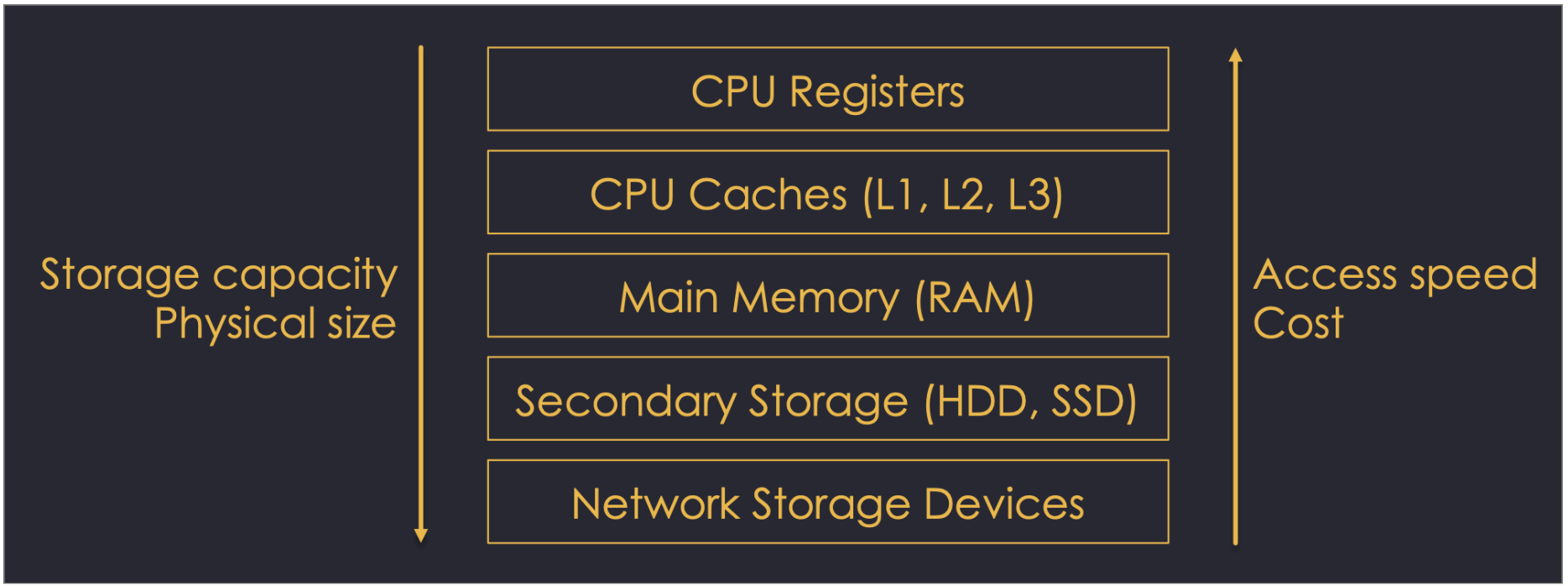
Memory + Static Type Systems



Why do modern computers have different types of memory?

The Memory Hierarchy

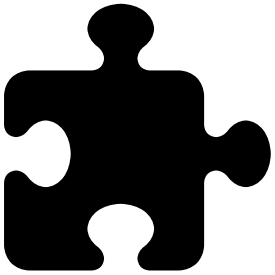
- Memory differs in size, access speed, physical location and cost
- Memory forms a hierarchy:



Caches

- Caches are storage buffers for frequently / recently used memory

Memory meets Physics



- Question 1: On a 3 GHz CPU, how much time does a single CPU cycle take?
- Question 2: Assume information travels at the speed of light and you want to read a byte from RAM, process it with a single instruction, and write it back to RAM at 3 GHz. What is the maximum distance between CPU and Memory that is physically possible?

Memory Size and Latency

- Memory that is arbitrarily large and arbitrarily fast is impossible: These two factors are mutually exclusive!

Memory Type	Typical Size	Typical access time
Register	A few bytes	One CPU cycle (less than a nanosecond)
L1 Cache	Dozens of KiBs	A few CPU cycles (about a nanosecond)
L2 Cache	Hundreds of KiBs	~10ns
L3 Cache	A few MiBs	20-40ns
Main Memory	GiBs	100-300ns
SSD	TiBs	A few hundred microseconds
HDD	Dozens of TiBs	A few milliseconds

Memory Type	Typical Size	Typical access time
Network Storage	Up to Exabytes	Hundreds of milliseconds to seconds

The Stack (recap)

- Special memory region & data structure for handling subroutine calls (i.e. functions)
- Needed for:
- Local variables
- Large arguments (that don’t fit in registers)
- One advantage of higher level languages: Automatic stack management

Scopes

- The region in the code where a given variable is valid is called the scope of that variable

```
1  fn foo() { // -------
2      let var = 42; // | *scope* of var (and also of fn foo)
3  } // ----------------
4
5  fn main() {
6      foo();
7      println!("{var}"); // Does not compile, var does not exist
8  }
```

- C, C++, and Rust use curly braces {} to define scopes

Scopes and the stack

```
1  fn f3(arg1: &i32, arg2: &i32) {
2      let var_f3 = 42;
3      println!("f3: {} {} {}", arg1, arg2,
4          var_f3);
5  }
6
7  fn f2(arg: &i32) {
8      let var_f2 = 53;
9      f3(arg, &var_f2);
10     println!("f2: {} {}", var_f2, arg);
11 }
12
13 fn f1() {
14     let var_f1 = 64;
15     f2(&var_f1);
16     println!("f1: {}", var_f1);
17 }
```

- How does the stack look like in line 3? And in line 10?

Stack memory is managed by the compiler!

- No need to write manual `push` and `pop` statements
- This is a form of automatic memory management!
- It has no overhead compared to manually writing `push` and `pop` (if your compiler is smart)
- Downsides:
- Stack size is limited (usually a few MiB)
- Stack memory is strictly hierarchical
- Allocations on the stack have to be know at compile time

Limitations of the stack

```rust
fn main() {
    let mut input = String::new();
    let mut numbers = ???;
    loop {
        println!("Insert a number or leave empty to finish:");
        std::io::stdin().read_line(&mut input)
            .expect("Failed to read input");
        if input.is_empty_or_whitespace() {
            break;
        }

        let number = input.parse::<i32>().expect("invalid value");
        numbers.push(number);
    }

    numbers.sort();
    println!("{numbers}");
}
```

Limitations of the stack

```
fn main() {
    let mut input = String::new();
    let mut numbers = ???;
    loop {
        println!("Insert a number or leave empty to finish:");
        std::io::stdin().read_line(&mut input)
            .expect("Failed to read input");
        if input.is_empty_or_whitespace() {
            break;
        }

        let number = input.parse::<i32>().expect("invalid value");
        numbers.push(number);
    }

    numbers.sort();
    println!("{numbers}");
}
```

- How many numbers will there be?
- How big will the `String` be?

Stack-only code is possible

```rust
const MAX_STRING_LEN: usize = 256;
const MAX_NUMBERS: usize = 128;

fn stoi(bytes: &[u8]) -> i32 { todo!() }

fn main() {
    let mut input: [u8; _] = [0; MAX_STRING_LEN];
    let mut input_size: usize = 0;
    let mut numbers: [i32; _] = [0; MAX_NUMBERS];
    let mut numbers_size: usize = 0;
    loop {
        println!("Insert a number or leave empty to finish:");
        input_size = std::io::stdin()
            .read(&mut input)
            .expect("Failed to read input");
        if input_size == 0 {
            break;
        }

        let number = stoi(&input[..input_size]);
        numbers[numbers_size] = number;
        numbers_size += 1;
    }

    numbers.sort();
    println!("{:?}", &numbers[..numbers_size]);
}
```

- Your task (in groups): Find as many things as you can that are wrong with this code!

- You can use LLMs, but group into “problems I understand” and “problems I don’t understand”
- 10 minutes

Dynamic memory allocations

- We need growable arrays:

```rust
fn main() {
    let mut input = String::new();
    let mut numbers = Vec::new();
    loop {
        println!("Insert a number or leave empty to finish:");
        std::io::stdin().read_line(&mut input)
            .expect("Failed to read input");
        if input.is_empty_or_whitespace() {
            break;
        }

        let number = input.parse::<i32>().expect("invalid value");
        numbers.push(number);
    }

    numbers.sort();
    println!("{numbers}");
}
```

A look into `Vec<T>`

- `Vec<T>` (or `std::vector<T>` in C++) manages a dynamic piece of memory
- The allocation is dynamic: It happens on-demand
- The size is dynamic: It can grow and shrink

Open questions

- How is memory allocated?
- How is it freed? When is it freed?
- How do uninitialized elements work?
- What happens if the memory on the heap is full?

Open questions

- How is memory allocated? -> unsafe Rust
- How is it freed? When is it freed? -> RAII
- How do uninitialized elements work? -> unsafe Rust
- What happens if the memory on the heap is full? -> unsafe Rust

Automatic deallocation?

- `numbers` goes out of scope at the end of main
- This cleans up the stack memory. What about the heap?

```rust
fn main() {
    let mut input = String::new();
    let mut numbers = Vec::new();
    loop {
        println!("Insert a number or leave empty to finish:");
        std::io::stdin().read_line(&mut input)
            .expect("Failed to read input");
        if input.is_empty_or_whitespace() {
            break;
        }

        let number = input.parse::<i32>().expect("invalid value");
        numbers.push(number); // Will allocate
    }

    numbers.sort();
    println!("{numbers}");
}
```

The `Drop` trait - Destructors in Rust

- Tell the Rust compiler: “Call this function when an instance of this type goes out of scope!”
- Allows us to clean up more than just the stack memory
- Mandatory whenever we write a type that owns a resource:
- `Vec<T>` owns a dynamic array on the heap
- `String` also owns a dynamic array (of UTF-8 bytes) on the heap
- `File` owns a file handle from the OS

The `Drop` trait - Example

```
impl<T> Drop for Vec<T> {
    fn drop(&mut self) {
        todo!("deallocate the buffer")
    }
}

fn main() {
    let mut numbers = Vec::new();
    numbers.push(42);
}
```

The `Drop` trait - Example

```
impl<T> Drop for Vec<T> {
    fn drop(&mut self) {
        todo!("deallocate the buffer")
    }
}

fn main() {
    let mut numbers = Vec::new();
    numbers.push(42);
    Drop::drop(&mut numbers);
}
```

- Compiler inserts the `drop` call
- Now we can never forget to call `free`/`delete` anymore!

RAII

- This is called resource acquisition is initialization (RAII)
- Terrible name for an amazing concept
- Better name: Scoped-based resource management
- Acquire a resource during initialization of the type
- Release the resource during destruction of the type

Is it really that simple?

```rust
fn print_numbers(numbers: Vec<i32>) {
    for number in numbers { println!("{number}"); }
}

fn main() {
    let mut numbers = Vec::new();
    loop {
        // Read numbers from command line...
    }

    print_numbers(numbers);
    numbers.sort();
    print_numbers(numbers);
}
```

Is it really that simple?

```
1  fn print_numbers(numbers: Vec<i32>) {
2      for number in numbers { println!("{number}"); }
3  }
4
5  fn main() {
6      let mut numbers = Vec::new();
7      loop {
8          // Read numbers from command line...
9      }
10
11     print_numbers(numbers);
12     numbers.sort();
13     print_numbers(numbers);
14 }
```

```
borrow of moved value: `numbers`
```

Semantics

- What does it mean to pass a value into a function?
- `print_numbers(numbers)`
- How does the function get access to the value?
- The CPU only knows bytes…
- Depends on the semantics of the programming language
- Semantics: The study of meaning in languages

Passing values to functions

- We have three options:

1. Copy
2. Move
3. Indirection (i.e. pointers)

Rust has move-semantics

- Rust moves values on assignment and function call
- A moved-from value becomes invalid

```
fn main() {
    let mut numbers = Vec::new();
    // ...
    print_numbers(numbers);
    numbers.sort(); // <- `numbers` has been moved!
    print_numbers(numbers);
}
```

- Why would we choose move-by-default?

Indirection by default?

A problem with copies

Copying resource owners

- We can fix our issues by doing a deep copy
- How is that performance-wise?
- What if an element on the heap is another resource-owner?
- How deep does a deep copy have to be?

Performance tradeoff

- Copy-by-default:
  - Hidden performance hits possible
  - Straightforward, doesn't "steal" anything
- Move-by-default:
  - Best performance
  - Transfers ownership (unusual?)
- Indirection:
  - On primitive types: Overkill
  - On large resource owners: Good

Primitive types and move

- Does this code compile?

```
fn print(number: i32) {
    println!("{number}");
}

fn main() {
    let num = 1024;
    print(num); // `num` moved into `fn print`
    print(num); // ?
}
```

- It does, but why?

The `Copy` trait

- A move has no direct equivalent on the hardware level
- It can be a copy
- The copy can be elided (e.g. return values)
- Registers might be reused for efficiency
- Primitive types always fit into registers
- This makes them cheap to copy
- Rust has a marker trait called `Copy` for types that support bitwise copies
- All primitive types implement `Copy`

`Copy` turns moves into copies

```
fn print(number: i32) {
    println!("{number}");
}

fn main() {
    let num = 1024; // i32 implements `Copy`
    print(num); // `num` NOT moved but COPIED!
    print(num); // `num` still valid :)
}
```

Who can implement `Copy`?

- Only fully self-contained types
- No indirection!
- The whole value of the type must live entirely on the stack
- What if we want deep copies instead?

Manually copying with the `Clone` trait

- We use the `Clone` trait!

```
trait Clone: Sized {
    fn clone(&self) -> Self;
}

impl<T: Clone> Clone for Vec<T> {
    //  ^---- deep copies require transitive Clone!
    fn clone(&self) -> Self {
        let mut cloned = Self::new();
        for value in &self {
            cloned.push(value.clone());
        }
        cloned
    }
}
```

`Copy`, `Clone`, moving and pinning

Come up with a `Copy + Clone` type, a `Clone` type, and a move-only type

Work in groups, duplicated types cancel out!

- From growable arrays to their limitations:
- Don’t forget to clean up memory (leads to RAII)
- Indirection / resource ownership changes value semantics (leads to Copy, Clone, move-by-default)

Speaker notes

